pip install numpy

In [2]:
```
#import this library in j.notebook
import numpy as np
```

creating an array using numpy

In [4]:
```
food = np.array(["pakora", "raita", "samoosa"])
food
```

Out[4]:
```
array(['pakora', 'raita', 'samoosa'], dtype='<U7')
```

In [5]:
```
price = np.array([5,5,5])
price
```

Out[5]:
```
array([5, 5, 5])
```

In [6]:
```
type(price)
```

Out[6]:
```
numpy.ndarray
```

In [7]:
```
type(food)
```

Out[7]:
```
numpy.ndarray
```

In [8]:
```
len(food)
```

Out[8]:
```
3
```

In [9]:
```
price[2]
```

Out[9]:
```
np.int64(5)
```

In [10]:
```
price[0:]
```

Out[10]:
```
array([5, 5, 5])
```

In [11]:
```
food[1]
```

Out[11]:
```
np.str_('raita')
```

In [12]:
```
price.mean()
```

Out[12]:
```
np.float64(5.0)
```

In [14]:
```
np.zeros(6)
```

Out[14]:
```
array([0., 0., 0., 0., 0., 0.])
```

In [15]:
```
np.ones(5)
```

Out[15]:
```
array([1., 1., 1., 1., 1.])
```

In [16]:
```
np.empty(5)
```

Out[16]:
```
array([1., 1., 1., 1., 1.])
```

In [18]:
```
np.arange(10)
```

Out[18]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [19]:
```
np.arange(5)
```

Out[19]:
```
array([0, 1, 2, 3, 4])
```

In [20]:
```
np.arange(2,20)
```

Out[20]:
```
array([ 2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
       19])
```

In [21]:
```
# specific interval
np.arange(2,20, 2)
```

Out[21]:
```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18])
```

In [23]:
```
# table
np.arange(0,55,5)
```

Out[23]:
```
array([ 0,  5, 10, 15, 20, 25, 30, 35, 40, 45, 50])
```

In [26]:
```
# line space
np.linspace(0,100, num=100)
```

Out[26]:
```
array([  0.        ,   1.01010101,   2.02020202,   3.03030303,
         4.04040404,   5.05050505,   6.06060606,   7.07070707,
         8.08080808,   9.09090909,  10.1010101 ,  11.11111111,
        12.12121212,  13.13131313,  14.14141414,  15.15151515,
        16.16161616,  17.17171717,  18.18181818,  19.19191919,
        20.2020202 ,  21.21212121,  22.22222222,  23.23232323,
        24.24242424,  25.25252525,  26.26262626,  27.27272727,
        28.28282828,  29.29292929,  30.3030303 ,  31.31313131,
        32.32323232,  33.33333333,  34.34343434,  35.35353535,
        36.36363636,  37.37373737,  38.38383838,  39.39393939,
        40.4040404 ,  41.41414141,  42.42424242,  43.43434343,
        44.44444444,  45.45454545,  46.46464646,  47.47474747,
        48.48484848,  49.49494949,  50.50505051,  51.51515152,
        52.52525253,  53.53535354,  54.54545455,  55.55555556,
        56.56565657,  57.57575758,  58.58585859,  59.5959596 ,
        60.60606061,  61.61616162,  62.62626263,  63.63636364,
        64.64646465,  65.65656566,  66.66666667,  67.67676768,
        68.68686869,  69.6969697 ,  70.70707071,  71.71717172,
        72.72727273,  73.73737374,  74.74747475,  75.75757576,
        76.76767677,  77.77777778,  78.78787879,  79.7979798 ,
        80.80808081,  81.81818182,  82.82828283,  83.83838384,
        84.84848485,  85.85858586,  86.86868687,  87.87878788,
        88.88888889,  89.8989899 ,  90.90909091,  91.91919192,
        92.92929293,  93.93939394,  94.94949495,  95.95959596,
        96.96969697,  97.97979798,  98.98989899, 100.        ])
```

In [27]:
```
np.linspace(1,10, num=5)
```

Out[27]:
```
array([ 1.  ,  3.25,  5.5 ,  7.75, 10.  ])
```

In [29]:
```
 np.ones(50, dtype=np.int64)
```

Out[29]:
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1])
```

In [30]:
```
np.ones(50, dtype=np.float64)
```

Out[30]:
```
array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

array functions

In [34]:
```
a = np.array([10,12,15, 2,4,6,100,320,0,5,10,3])
a
```

Out[34]:
```
array([ 10,  12,  15,   2,   4,   6, 100, 320,   0,   5,  10,   3])
```

In [36]:
```
a.sort()
a
```

Out[36]:
```
array([  0,   2,   3,   4,   5,   6,  10,  10,  12,  15, 100, 320])
```

In [39]:
```
c = np.array([10.2,3.4,53.6,91.6,45.5])
c
```

Out[39]:
```
array([10.2,  3.4, 53.6, 91.6, 45.5])
```

In [40]:
```
b= np.concatenate((a,b))
b
```

Out[40]:
```
array([  0. ,   2. ,   3. ,   4. ,   5. ,   6. ,  10. ,  10. ,  12. ,
        15. , 100. , 320. ,  10.2,   3.4,  53.6,  91.6,  45.5])
```

In [41]:
```
c.sort()
c
```

Out[41]:
```
array([ 3.4, 10.2, 45.5, 53.6, 91.6])
```

2-D arrays

In [53]:
```
x = np.array([[1,2], [5,4]])
x
```

Out[53]:
```
array([[1, 2],
       [5, 4]])
```

In [54]:
```
y = np.array([[6,7],[7,8]])
y
```

Out[54]:
```
array([[6, 7],
       [7, 8]])
```

In [59]:
```
np.concatenate((x,y), axis=1)
```

Out[59]:
```
array([[1, 2, 6, 7],
       [5, 4, 7, 8]])
```

In [58]:
```
np.concatenate((x,y), axis=0)
```

Out[58]:
```
array([[1, 2],
       [5, 4],
       [6, 7],
       [7, 8]])
```

In [63]:
```
a = np.array([[[0,1,2,3],
               [4,5,6,7]],
              [[0,1,2,3],
               [4,5,6,7]],
             [[0,1,2,3],
               [4,5,6,7]]])
a
```

Out[63]:
```
array([[[0, 1, 2, 3],
        [4, 5, 6, 7]],

       [[0, 1, 2, 3],
        [4, 5, 6, 7]],

       [[0, 1, 2, 3],
        [4, 5, 6, 7]]])
```

In [64]:
```
# to find the number of dimensins
a.ndim
```

Out[64]:
```
3
```

In [67]:
```
b = np.array([[5,6,7],
              [8,9,10],

              [10,11,12]])
b
```

Out[67]:
```
array([[ 5,  6,  7],
       [ 8,  9, 10],
       [10, 11, 12]])
```

In [68]:
```
b.ndim
```

Out[68]:
```
2
```

In [72]:
```
type(a)
```

Out[72]:
```
numpy.ndarray
```

In [ ]:
```
b = np.array([[5,)
```

In [73]:
```
b = np.array([[5,6,7],
              [8,9,10],

              [10,11,12]])
b
```

Out[73]:
```
array([[ 5,  6,  7],
       [ 8,  9, 10],
       [10, 11, 12]])
```

In [74]:
```
a.shape
```

Out[74]:
```
(3, 2, 4)
```

In [ ]:

In [82]:
```
a = np.arange(9)
a
```

Out[82]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8])
```

In [83]:
```
b = a.reshape(3,3)
b
```

Out[83]:
```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

In [88]:
```
# reshape
np.reshape(a, newshape=(1, 9),order='c'
               a
```

```
  Cell In[88], line 2
    np.reshape(a, newshape=(1, 9),order='c'
               ^
SyntaxError: '(' was never closed
```

In [89]:
```
#  convet 1D into 2D
a = np.array([1,2,3,4,5,6,7,8,9])
a
```

Out[89]:
```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [ ]:

In [90]:
```
a.shape
```

Out[90]:
```
(9,)
```

In [92]:
```
b = a[np.newaxis, :]
b
```

Out[92]:
```
array([[1, 2, 3, 4, 5, 6, 7, 8, 9]])
```

In [94]:
```
b.shape
```

Out[94]:
```
(1, 9)
```

In [95]:
```
c = a[np.newaxis, :]
b
```

Out[95]:
```
array([[1, 2, 3, 4, 5, 6, 7, 8, 9]])
```

In [96]:
```
c = a[: , np.newaxis]
c
```

Out[96]:
```
array([[1],
       [2],
       [3],
       [4],
       [5],
       [6],
       [7],
       [8],
       [9]])
```

In [97]:
```
a[2:6]
```

Out[97]:
```
array([3, 4, 5, 6])
```

In [98]:
```
a*5
```

Out[98]:
```
array([ 5, 10, 15, 20, 25, 30, 35, 40, 45])
```

In [99]:
```
a+5
```

Out[99]:
```
array([ 6,  7,  8,  9, 10, 11, 12, 13, 14])
```

a.sum()

In [100…
```
a.mean()
```

Out[100…
```
np.float64(5.0)
```

In [ ]:
```

```